TANSZÉKVEZETŐ

SZAKDOLGOZAT FELADAT

Csutár Márk Tibor
Mérnökinformatikus hallgató részére

Orvosi adatok feldolgozása gépi tanulással

A mélytanulás lehetővé teszi, hogy orvosi adatokon olyan elváltozásokat is felismerjünk, amiket a tradicionális módszereken túlmutatnak. Azonban az orvosi adatok feldolgozása speciális körültekintést igényelnek, nem lehet a képfeldolgozásban alkalmazott neurális hálókat egy az egyben átültetni. Nagyon fontos az előfeldolgozás, illetve az adatoknak a kiértékelése.

A hallgató feladata ilyen orvosi, mélytanulás alapú algoritmusok tervezése, megvalóstása és kiértékelése. A hallgató munkája során elsősorban melanóma felvételekkel dolgozik.

A hallgató feladatának a következőkre kell kiterjednie:

- Mutassa be speciálisan az orvosi adatokat! Elemezze az adatokat!
- Ismertesse a képfeldolgozásnál használt neurális háló architektúrákat! Csoportosítsa ezeket! Készítsen összehasonlítást előnyeikről és hátrányaikról!
- Tervezzen és implementáljon neurális háló architektúrát orvosi adatok feldolgozására!
- Tesztelje az architektúrát és elemezze az eredményeket!

Tanszéki konzulens: Dr. Szegletes Luca, egyetemi adjunktus

Budapest, 2026. 03. 16.

Dr. Charaf Hassan
egyetemi tanár
tanszékvezető

Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar
Automatizálási és Alkalmazott Informatikai Tanszék

H-1117 Budapest, Magyar tudósok krt 2. Q.B.207
Telefon: 463-2870. Fax: 463-2871
www.aut.bme.hu

Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar
Automatizálási és Alkalmazott Informatikai Tanszék

Csutár Márk Tibor

ORVOSI ADATOK FELDOLGOZÁSA GÉPI TANULÁSSA

KONZULENS

Dr. Szegletes Luca

BUDAPEST, 2026

Tartalomjegyzék

Összefoglaló 5
Abstract 6
1 Bevezetés 7
2 Irodalomkutatás és technológiák 8
2.1 Klasszikus gépi képfelismerés 8
2.2 Kép előfeldolgozási technikák 9
2.2.1 Általános előfeldolgozási lépések 9
2.2.2 Kernel konvolúció 9
2.3 Neurális hálók képfelismerésre 10
2.3.1 Konvolucionális Neurális Hálók 10
2.4 Transformerek képfelismerésre 13
2.4.1 Eredeti transzformer architektúra 13
2.4.2 Vision Transformer 17
2.4.3 Swim Transzformer: 18
2.5 Hibrid modellek 20
3 Tervezés 22
3.2 Választott adathalmazok 22
3.3 Választott technológiák 22
3.3.1 Számítási platform 22
3.3.2 Programnyelv és keretrendszerek 23
3.3.3 Explainable AI 23
4 Irodalomjegyzék 24
Függelék 26
4.1 Függelék 26
Nyilatkozat generatív mesterséges intelligencia alkalmazásáról 26

HALLGATÓI NYILATKOZAT

Alulírott **Csutár Márk Tibor**, szigorló hallgató kijelentem, hogy ezt a szakdolgozatot/ diplomatervet meg nem engedett segítség nélkül, saját magam készítettem, csak a megadott forrásokat (szakirodalom, eszközök stb.) használtam fel. Minden olyan részt, melyet szó szerint, vagy azonos értelemben, de átfogalmazva más forrásból átvettem, egyértelműen, a forrás megadásával megjelöltem.

Hozzájárulok, hogy a jelen munkám alapadatait (szerző, cím, angol és magyar nyelvű tartalmi kivonat, készítés éve, konzulens(ek) neve) a BME VIK nyilvánosan hozzáférhető elektronikus formában, a munka teljes szövegét pedig az egyetem belső hálózatán keresztül (vagy hitelesített felhasználók számára) közzétegye. Kijelentem, hogy a benyújtott munka és annak elektronikus verziója megegyezik. Dékáni engedéllyel titkosított diplomatervek esetén a dolgozat szövege csak 3 év eltelte után válik hozzáférhetővé.

Kelt: Budapest, 2026. 0X. XX

...…….

Csutár Márk Tibor

Összefoglaló

Ezt majd a végén

Abstract

This too.

1 Bevezetés

2 Irodalomkutatás és technológiák

Melanoma kiszűrésében és kezelésében az egészségügy is erősen támaszkodik a páciensek önellenőrzésére, amitől a gyanú felmerül. A laikus szem azonban szakavatatlan, nem feltétlen tudja mire kell figyelni. E dolgozatban célom, hogy a számítógép segítséget nyújtson ebben, és egy bőrelváltozásról készült fénykép alapján, hatékonyan és a lehető legpontosabban el tudja dönteni, hogy a képen melanoma vagy más bőrelváltozás látható-e.

A megvalósításhoz azonban nem mindegy, a számítástechnika mely technológiát használjuk. A következő fejezetben áttekintést szeretnék adni, milyen technológiák használhatóak erre a feladatra, tudományos hátterükről és hogyan érdemes gyakorlati használatba ültetni őket.

2.1 Klasszikus gépi képfelismerés

A gépi tanulás ma ismert fellendülése előtt is sok próbálkozás volt olyan programok írására, ami megmondja, mi szerepel egy képen. Működésük bemutatása nélkül, pár sikeres algoritmus, hogy kontextusba helyezhessük a ma elterjedt technológiákat:

- **Scale-invariant feature transform (1999):** Objektumok megtalálására működött jól, az objektum méret, forgatás, kisebb perspektíva változása mellett is. De kategorizálásra nem valamint magas a számítás igénye.
- **Histogram of oriented gradients (~1994):** Egyszerűbb, gyorsabb és hatékonyabb. Jól működött ha az objektumot az élei jól körülhatárolták; Például ember detektálásban pontos. Háttérzaj, az objektumok elrendezése, pózok, képtméret változására érzékeny.
- **Viola–Jones detector (2001):** Mérföldkőnek tekinthető. Nagyon gyors, alacsony az erőforrás igénye (valós időben is használható volt a korabeli hardveren). Egyszerűbb, jól struktúrált objektumok detektálásában, frontális arcdetektálásban jó. Nézőpont változására, takarásra azonban érzékeny volt.

Ezek a klasszikus gépi látási módszerek eredményeik ellenére korlátozottnak bizonyultak. Bonyolult alakzatú objektumokat pontatlanul detektáltak, kézzel kellett megadni, hogy a képek részleteit hogyan érzékeljék, érzékenyek voltak zajra. Végképp nem értelmeztek szemantikát és kontextust a képeken. Ezekben sokkal jobbnak bizonyultak a mélytanuláson alapuló képfelismerő architektúrák. [1]

2.2 Kép előfeldolgozási technikák

Egy programnak, ami bemenetül egy képet kap és megmondja, mi van rajta, a legritkább esetben ideális bemenet az eredeti nyers kép, ezért a bementi kép szinte minden esetben átmegy egy előfeldolgozáson. Előfeldolgozásra a számítási igény csökkentésén kívül azért is szükség lehet, hogy kiemeljük a számunkra releváns részleteket, elhanyagoljuk a szükségteleneket valamint a számokkal dolgozó gép számára feldolgozhatóbbra fordítsuk azokat. Ahogy látni fogjuk, a legtöbb képfelismerő technológia nem önmagában való, hanem alapul veszi képek előfeldolgozásának valamilyen kombinációját. [2]

2.2.1 Általános előfeldolgozási lépések

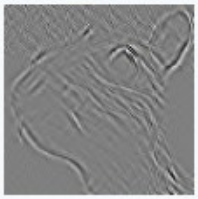
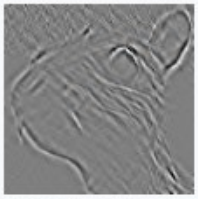
- **Átméretezés:** A neurális hálók fix méretű bemenetet várnak. Ez gyakran társulhat a kép kiegészítésével üres pixelekkel, hogy ne veszítsünk információt skálázás okozta torzítással, vagy eltérő képarányú képeknél.
- **Színtér-átalakítás:** Ha a színek nem, csak az objektumok szerkezete fontos, elég a fekete-fehér. De ahol relevánsak a színek sem mindig az RGB a legjobb reprezentáció.
- **Normalizálás:** Ebben a lépésben a képeket egységes tartományba hozzák. Például 0-255 közötti RGB értékek helyett 0-1 közötti számra. Mivel a modell számokkal dolgozik ezzel a lépéssel stabilabb működés érhető el.
- **Adat kiegészítés:** Nem szigorúan előfeldolgozás. Ebben a lépésben mesterséges változtatásokat (kitakarás, forgatás) alkalmazunk a képekre. A tanító képeken lehet érdemes, hogy a modell jobban tudjon álltalánosítani.

2.2.2 Kernel konvolúció

A kernel konvolúció egy önállóan is hasznos technológia, mesterséges intelligenciáktól függetlenül, például képmanipulációra (elmosás, él kiemelés). Ereje abban rejlik, hogy a pixeleket a szomszédaikkal együtt vizsgáljuk. A részletek (él, sarok) pont a lokális kapcsolatokban, a képpontok szomszédaikhoz képesti megváltozásában rejlenek (szín, világosság megváltozása).

Működése a következő: A képen -amit szám mátrixként értelmezünk-, egy kisebb mátrxot végig csúsztatunk minden egyes képponton. Ez a kisebb mátrix a kernel. A kép minden pixelére a környező pixelek kernel értékekkel súlyozott összegét számoljuk ki.

A kernel értékektől függően nagyon sok oldalúan használható.

Operation	Kernel ω	Image result g(x,y)
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Ridge or edge detection	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	

1. ábra Példák különböző kernel értékek eredményeire [4]

2.3 Neurális hálók képfelismerésre

Neurális hálók a gépi tanulás egy lehetséges megvalósításának családja sok más módszer közül. A módszerek között pár fő különbség, hogy milyen adatokon működnek jól és mennyire erőforrás igényesek. Egyszerű mintákra vannak sokkal hatékonyabb módszerek, a képfeldolgozásban azonban bonyolult mintázatokat kell felismerni.

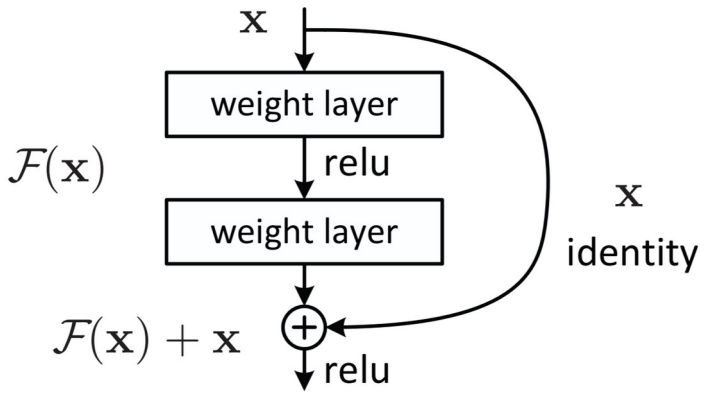
Nem is véletlen, hogy bár a kalsszikus algoritmusok is tartalmaztak valamilyen gépi tanulást, azok a bonyolult mintákon korlátozottak voltak, így lényegében letarolták a képfeldolgozást a neurális hálók. További előnyük a klasszikus módszerekhez képes, hogy amíg azoknak kézzel kellett megadni, hogyan találják meg és dolgozzák fel a képek részleteit és az algoritmus végén volt egy osztályozó ami tanult; addig a neurális hálók elejüktől végükig, pusztán adaton, automatikusan tanulták meg, mely részletek relevánsak.

2.3.1 Konvolucionális Neurális Hálók

Angolul: Convolutional Neural Networks (továbbiakban: CNN). Motorjuk a már részletezett kernel konvolúció, azzal a különbséggel, hogy a kernel értékeit nem kézzel adjuk meg (az esetleges kezdeti értékeket kivéve), hanem a modell tanulja.

CNN-ek tipikus alkotó elemei:

- **Konvolucionális rétegek**: Ezek kernel konvolúciót végeznek, aminek eredménye egy olyan aktivációs réteg (feature map), ami olyan mintázatokat tartalmaz amiket a háló hasznosnak talál. Ilyen rétegekből több is előfordulhat, a modell elején az egyszerűbb, későbbi rétegekben komplexeb részletek észlelésére.
- **Pooling réteg**: Jellemzően a konvolúciós rétegeket követően. Mivel a detektált minták (él, sarok, stb.) az eredeti képhez képest sokkal kevesebb pixelen is értelmezhetőek. Pixelek egy csoportját egy beléjük táplált függvény alapján összevonja (pl.: max pooling esetén egy pixal ablak legnagyobb értékét adja tovább az új, 1db pixelnek). Alkalmazásukkal számítási teljesítmény és eltolásokkal szembeni robusztusság is nyerhető. Ugyanakkor nem minden architektúra használja, hasonló nyereség azzal is elérhető, ha kernel konvolúció során a kernelt 1 helyett több pixellel mozdítjuk odébb.
- **Előre csatolt kapcsolatok** (residual connections): Egy réteg kimenetét nemcsak az azt követő, hanem egy valahánnyal későbbi rétegnek közvetlenül is előre csatoljuk. Segíti, hogy az eredeti bemenet ne vesszen el teljesen, egyes minták később is hasznosak lehetnek. Mély hálók tanítását és használatát segíti.



2. ábra: Átugró kapcsolatok [7]

- **Osztályozó réteg**: Ez a gyakori elnevezés vagy fully-connected layer, de valójában azt csinálja, amire be van tanítva ami nem csak osztályozásban merül ki. Ez egy tipikus, neurális háló ami a kimenetet adja.

Néhány meghatározó CNN architektúra:

- Visual Geometry Group hálói (VGGNet): CNN architektúrák egy csoportja, VGG+rétegek száma elnevezési konvencióval (VGG16, VGG19). 2014-ben vált

ismerté az ImageNet Large Scale Visual Recognition Challenge-en (továbbiakban: ILSVRC) elért 92.7%-os pontosságával.

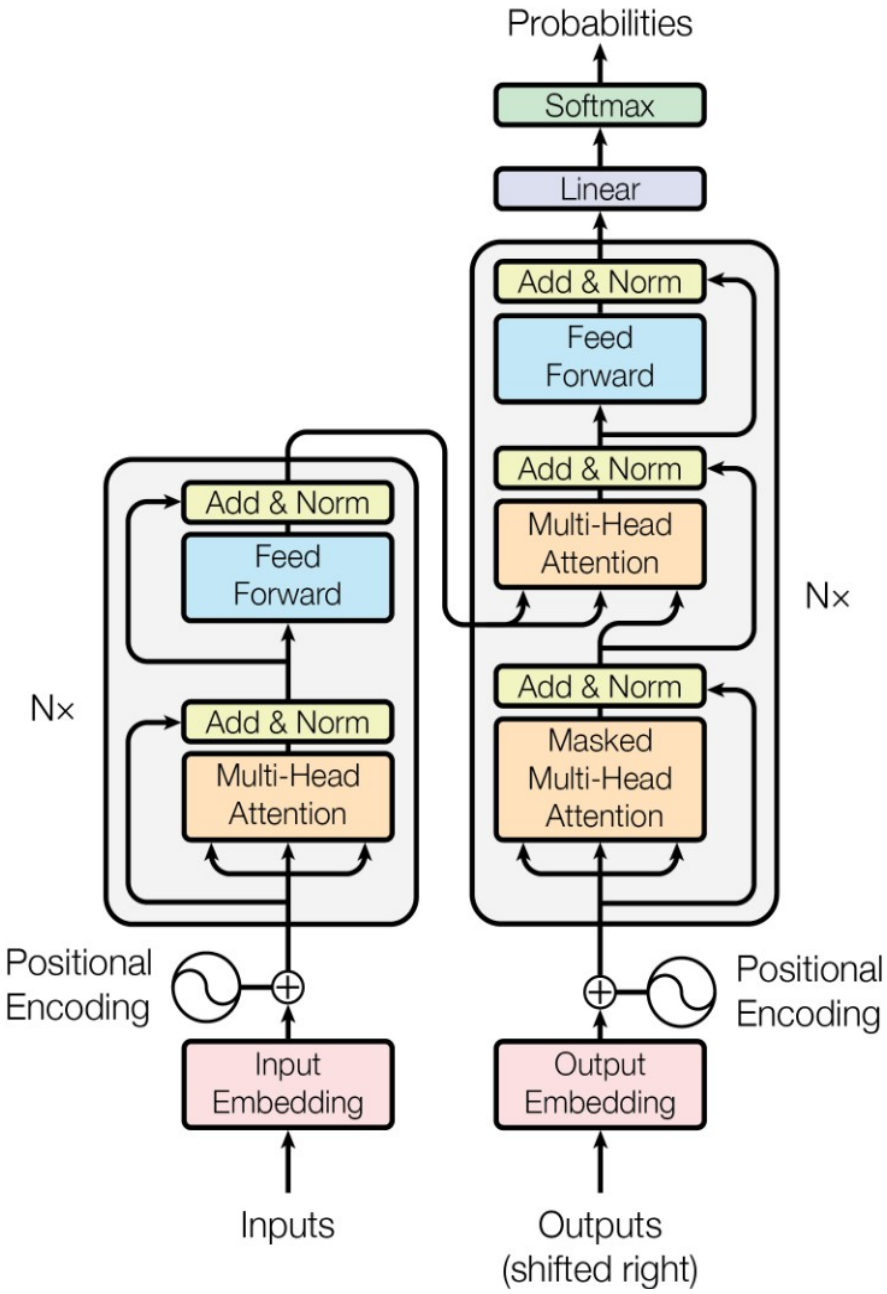
- Residual Networks (Előrecsatolt hálók, resnets): 2015-ben jelentek meg és meg is nyerték az az évi ILSVRC-t. VGG hálók a méretük növelésével már nem értek el pontosabb működést és nehézzé vált a tanításuk. A residual hálók előrecsatolt kapcsolatokat vezettek be, ezzel megkönnyítva a tanítást és (sokkal) nagyobb méretű hálók létrehozását, amik így nagyobb pontosságot tudtak elérni.
- ConvNeXt: 2022-ben megjelent modern, jelenleg egyik legkorszerűbb CNN architektúra. A transzformerek megjelenésével (lásd: később) a CNN-ek háttérbe szorultak. A ConvNeXt megkísérli a CNN-ek versenyben tartását olyan, egyébként transzformerek által ihletett megoldásokkal mint: nagyobb kernel méret, réteg szintű normalizálás, előrecsatolt kapcsolatok. [8][9]

[3][7]

2.4 Transformerek képfelismerésre

2.4.1 Eredeti transzformer architektúra

Transzformer modellek 2017-ben lettek bemutatva, először gépi fordításra használva, de az architektúra mögötti koncepciók generikusak annyira, hogy ma már számos és jelentős feladatra is épült tanszformer alapú megoldás; Mint: Chatbot/szöveg generálás, audio és kép feldolgozás. Kép feldolgozásban a Vision Transformer (továbbiakban: ViT.) és az általa ihletett modellek ma az élen járók.



3. ábra: Eredeti transzformer architektúra az Attention is All you Need-ből [5]

Transzformerek kulcs újítása a **attention** mechanizmus, a bemenet egészén vizsgálja, hogy az egyes részek (tokenek) mennyire relevánsak a többi számára. Mondhatni, összebeszélnek a tokenek. Szöveges esetben egy szóra mennyire hatnak a korábbi szavak; például: „A cica aki egerészett gyors volt!” mondatban a „cica” és „gyors” szavak kapcsolódnak, még annak ellenére is, hogy a mondat 2 végén vannak. Ebben a cicás példában a gyors mint melléknév a cica egy tulajdonságát jelenti, de persze szavak máshogyan is hathatnak egymásra. Ilyen lehet (sok máson kívül) például nyelvtani kapcsolat (alany-állítmány, melléknév-főnév), névmások és hozzájuk tartozó entitások feloldása, idéző- és zárójelek összekapcsolása és jelentésbeli (szemantikai) kapcsolat.

Fontos kiemelni, hogy a transzformer modellek által feldolgozott alapegység a **token**. Szöveges esetben nem feltétlenül teljes szó, inkább szótő vagy toldalék. Egy ilyen szó-részt egy sok-sok dimenziós geometriai térbe ágyazással vektorrá alakítanak. Képek esetében pedig 1-1 kép-szeletet (patchet) ágyaznak tokenbe.

Tokenizáció pontos folyamata szöveges esetben itt most nem releváns, képek esetén pedig Vision Transformerek fejezetben fogom bemutatni, viszont a geometriai térbe egyazás miértje itt is fontos. Erre a reprezentációra azért van szükség, mert így a szavak (vagy amit beágyazunk) jelentése válik számíthatóvá. Egy klasszikus példa ezt jól szemlélteti: Képzeljük el, hogy a férfi, nő illetve király királynő szavakat ábrázoljuk irányvektorokkal, majd ebben az elképzeld koordináta rendszerben vonjok ki a nőből a férfit. Ennek az eredménye egy új vektor, ami a férfi vektor végétől a nő vektor végébe mutat, ha ezt a vektort pedig hozzáadjuk a király vektorhoz, meg kell kapnunk a királynő vektort. Mintha ebben a geometriai térben az egyik irány nemiség jelentést hordozna.

A transzformer modellek azon része, ami a tokenek egymásra hatását számítja az ugynevezett **attention head** (3. ábrán: Multi-Head attention). Legintuitívabban úgy lehet elképzelni, egy attention head belső működését, mintha ez a kérdés szerepelne benne: „Ha itt ilyen token van, mely korábbi tokenek relevánsak és mennyire?”. Egy attention head arra, hogy mi alapján hatnak egymásra a tokenek, egyszerre csak 1 mintát tud megtanulni, ezért több attention head szerepel egy modellben (innen a Multi-Head).

Itt azt is fontos megjegyezni, hogy azt, hogy mi alapján értékel 2 tokent összefüggőnek, azt a modell tanulja, ráadásul nulláról tanulja. Az említett összefüggőségi példák (nyelvtani, szemantikai, stb) feltűnése jellemző, de a tanulás során alakulnak ki, nem előzetes szándék alapján.

Az attention head-ek kimenete pedig lényegében egy módosítás az eredeti, bementi tokenen, ami a „jelentését” a kontextus ismeretében módosítja. Mivel ez a kimenet csak maga a módosítás, így a kimenetet előre csatoltan hozzá kell adni a bemeneti tokenhez (3. ábrán ilyen lépés az Add & Norm). Következő példán lehet ezt elképzelni: A fa szó más jelentéssel bír egy biológiai, mint egy számításelméleti könyvben, de a bementi szöveg tokenekre bontásakor viszont ugyanazt a vektort kapja. Az attention szándékozik kimenetéül adni azt az „eltolást” ami az eredeti „fa” vektort a szövegkörnyezet függvényében a „fa mint organizmus” vagy a „fa mint gráf” pozícióba tolja. Vagyis, a puszta szót vektorát a kontextus alapján további jelentéssel gazdagítsa.

Mivel több attention head szerepel egy modellben, amik különböző, de egyformán fontos részleteket tanulnak meg, a számításaik végén adott kimenetet össze kell fűzni. Ennek egy további óriási előnye, hogy emiatt az attention head-ek számításia párhuzamosan is végezhetők.

[5][8]

Attention számítása:

1. Bemenet: **E** tokenek
2. Legyen 3 mátrixunk, Query, Key, Value: $\mathbf{W^Q}$, $\mathbf{W^K}$, $\mathbf{W^V}$! A mátrixok értékei egyben a modell paraméterei, amik a tanulás során változnak, finomhangolódnak.
3. Midegyik mátrixot szorozzunk meg mindegyik $\mathbf{E_i}$ bementi tokennel, így állítva elő **q, k, v** vektorokat.

   $\mathbf{q_i = W^qE_i}$ $\mathbf{k_i = W^qE_i}$ $\mathbf{v_i = W^qE_i}$

4. Vegyük mindegyik **q** vektornak mindegyik **k** vektorral a skaláris szorzatát. Legyen ez az attention score.

   $\mathbf{score_{ij} = q_i \cdot k_j}$

   A skaláris szorzat értelmezhető egy vektor másikra vetett vetületeként, ami akkor nagy, ha a vektorok hasonló irányba néznik. Ez a hasonló irányba nézés, illeszkedés, felel meg a 2 token kapcsolódásának, hogy mennyire „attendálnak” egymásra.

5. Az attention score-kat normalizáljuk úgy, hogy minden oszlopa megfeleltethető legyen egy eloszlásnak, softmax képlet alkalmazásával, aminek tulajdonsága, hogy az elemek oszlop összege 1

$$\boldsymbol{\alpha_{ij} = (\exp(score_{ij})) / (\sum_j \exp(score_{ij}))}$$

Ekkor minden α érték 0-1 közötti valós szám és a belőlük alkotott mátrixot nevezhetjük **attention pattern**-nek.

6. Az **attention pattern** minden i-ik sorának minden elemét össze skalárszorozzuk az i-ik value vektorral.

$$\boldsymbol{z_{ij} = \alpha_{ij} v_j}$$

Ezek a z értékek már nem valós számok, hanem a value vaktorok lényegében súlyozva az attention pattern értékeivel. Vagyis a value vektorok súlyozva azzal, hogy a 2 token mennyire attendál egymásra.

7. Végeredményben a súlyozott value vektorok mátrixának oszlopait összegezzük.

$$\boldsymbol{\Delta E_i = \sum_j z_{ij}}$$

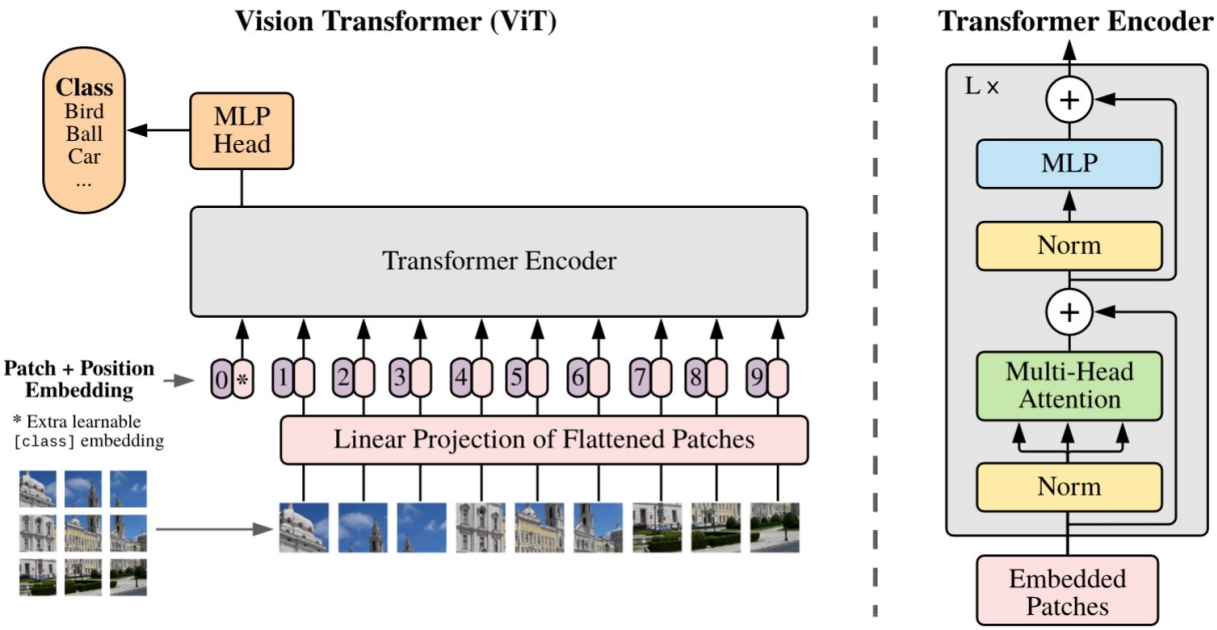
Az így kapott ΔE_i tartalmazza azt a módosítást, amit az eredeti i-ik $\mathbf{E_i}$ tokenhez adva, az annak jelentését a kontextus függvényében gazdagítja.

A fenti folyamatot a következőképpen foglalnám össze és fordítanám kicsivel emberibb nyelvre: 2 token illeszkedését, egymásra hatását a skaláris szorzás adja, viszont a skaláris szorzás operandusai nem közvetlenül a bemeneti tokenek, hanem azok szorzata a Query és Key mátrixokkal, mely mátrixok a modell paraméterei, ezáltal tanulja meg a modell, hogy összefüggő szavak esetén a mátrixok ugyanezekkel tokenekkel szorozva egymásra illeszkedő vektorokat kell adjanak. Miután pedig megvan, hogy mely token van hatással melyikre, a Value mátrixot a ható tokennel szorozva és súlyozva kapjuk meg azt a módosítást, amit hozzá kell adni a hatott tokenhez. És mivel a Value mátrix is a modell paramétereit tartalmazza, az, hogy egy token milyen módosításnak felel meg egy későbbin, a modell szintén tanulja.

Pongyola példa: „zöld erdő” szavakkal q=Query*erdő és k=Key*zöld után q·k megmondja, hogy „zöld” mennyire hat „erdő”-re, ennek ismeretében pedig v=Value*zöld egy „zöldítő” vektort ad, amit ha az „erdő” tokenjéhez adnánk, akkor az gazdagodna azzal a jelentéssel, hogy egy élő nyári zöld erdőr utal, nem pedig egy lombhullatott télire.

[8]

2.4.2 Vision Transformer



4. ábra: Vision Transformer modell arhitektúra [6]

Szöveges példák után pedig nézzünk a képekkel dolgozó ViT motorházteteje alá. A 4. ábrán látható arhitektúra elemeit részletezve.

I. **Bemeneti kép patch-ekre bontása**: A kép felszeletelése, n*n-es részekre. Ezeknek a képszeleteknek a mérete és ebből következően, hogy hány patch-re bomlik a kép céltól és erőforrások fényében változhat. Az eredeti cikkben 16*16-os patcheket használtak.

II. **Embedding**: Egy rgb kép esetén egy patch n*n pixel *3 szín csatorna. Kilapítással vektorrá alakítjuk (pl.: 16*16 pixel * 3 szín = 768 dimenziós vektor). Ezután hozzárendeljük, hogy a teljes képen hol helyezkedik el a patch. Erre azért van szükség, mert a párhuzamosított attention head-ek és az utánuk végzett összegűzés érzéketlen a patch-ek pozíciójára, így a modell későbbi részei számára elveszne, hogy amit vizsgál képrészlet pldául közepén vagy sarokban van. Ez a patch és pozíció vektor eggyütesen a token.

III. **Transformer encoder**: Kompozit blokk. A bemeneti beágyazott tokenek az alábbi lépéseken mennek kereszül.

   1. Tokenek normalizálása.

   2. Normalizált tokenek áteresztése egy több attention head-ből álló self attention rétegen.

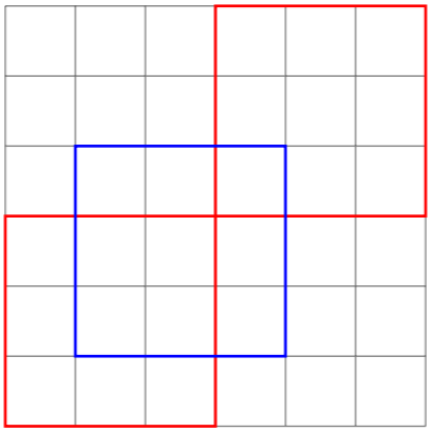
3. Az attention head-ek kimeneteinek és az előrecsatolt bemeneti tokenek összeadása.
4. Már self attention-t tartalmazó tokenek újbóli normalizálása.
5. Többrétegű perceptron (Multi-Layer Perceptron, MLP) réteg. Ez valójában egy sima, általában több rétegű neurális háló, hasonlóan a CNN-ek végén szereplő fully-connected réteghez. Ez a réteg tokenenként dolgozik, vagyis a tokenek itt már nem beszélnek össze, mint az attention-nél. Miután az attention gazdagította a tokeneket, hogyan kapcsolódnak egymáshoz. Ez a blokk absztraktabb mintákat tanul meg az egyes tokeneken. Egy cicás kép példáján az attention megtanulja, hogy az orr, fül, szem patch-ek kapcsolódnak és hogy egy arc részei; az MLP pedig pl. önállóan az orr patch-en a szín, forma, arc kontextus ismeretében kimenetül adja, hogy ez a patch valószínűleg egy cica orr.

IV. **Osztályozó**: Végül a transformer encoder kimenetét egy osztályozónak adjuk. Ez egy kis, legtöbbször 1 rétegű neurális háló, ami már a kívánt osztálycímkéket (vagy amire be van tanítva) adja kimenetül.

[6]

2.4.3 Swim Transzformer:

A részletezett Vision Transzformerre lehet úgy gondolni, mint az eredeti vision transzformerre. Azonban mivel ez a modell lényegében 1-1 átvétele az eredetileg szövegekre kitalált transzformer architektúrának, annak nagy számítási igényével, azóta jelentek meg további, optimalizáltabb ViT-ek, még inkább képek feldolgozására specializálódva. Ilyen például a Data-efficient Image Transformer (DeiT, 2021) vagy a Multi-Axis Vision Transformer (MaxViT, 2022). Az én feladatomhoz a Swim Transzformert találtam érdekesnek, ezt szeretném röviden ismertetni.

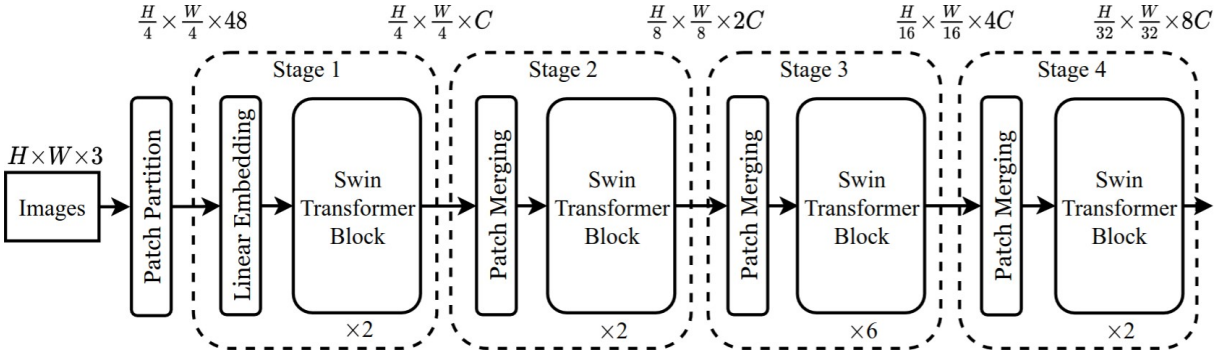


5. ábra: "Lokális ablakok, eltolással rétegenként" [8]

Az eredeti ViT-ben minden token (patch) figyelt mindegyik másra, emiatt a képméret növekedésével a számítási igény négyzetesen nő. A Swim Transformer ennek mérséklésére vezette be a csúsztatott ablakos (shifted window) technikát, hogy a képmérethez képest lineáris komplexitás növekedést érjen el.

Csúsztatott ablak esetén nem minden tokenre minden másikkal számít összefüggést, hanem csak 1, fix méretű ablakon belül beszélnek össze a tokenek és a képet több ilyen ablakkal fedi le. Ezzel azonban a képrészletek csak lokálisan, az ablakon belül figyelnének egymásra, ezért későbbi rétegekben elcsúsztatva, szándékosan a korábbi ablakokkal átfedő részekre is illeszt ablakot, így nyerve globális kontextust.

[8][11]

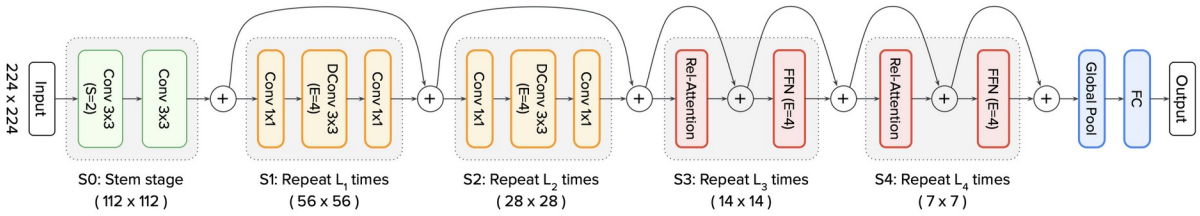


6. ábra: Swim Transformer architektúra [11]

2.5 Hibrid modellek

CNN-ek nagyon jól kezelik a lokális mintákat, kevés adaton is jól működnek, alacsony a komplexitásuk, viszont globális összefüggések megtanulására csak sok réteggel képesek. Transzformerek ezzel szemben azonnal az egész kép kontextusát figyelembe veszik, viszont alapból nem rendelkeznek lokális minta felismeréssel, ezért ezt is nulláról kell megtanulniuk ami következményeként rengeteg adat szükséges a tanításukhoz (adatéhesek) és a számítási igényük is nagyra tud nőni.

A két megközelítést kombinálni és így az előnyeik eggyütes megnyerését kísérlik meg a hibrid hálók. Számos ilyen modell létezik már (például: Conformer, MobileViT), ilyenek közül a Convolution + Attention hálót (CoAtNet) szeretném részletezni, ami nem az első próbálkozás a 2 technológia egyesítésére, viszont átfogóan kísérli meg annak helyes végrehajtását.



7. ábra: CoAtNet architektúra felépítése [10]

Az 7-ik ábrán látható CoAtNet háló az őt bemutató cikk-ben szemléltetett felépítése 2 koncolúciós és 2 attention réteggel (C-C-T-T), de a cikk számításba vesz és kísérletezik ettől eltérő konvolúció/attention aránnyal. Fontos, hogy a koncolúciót és az attention-t nem keveri, hanem először használ konvolúcióat az alacson szintű minták megtanulására és később attentiont a kép szintű összefüggések megtanulására.

A rétegek működése részletesebben:

- S0: Bemenet előfeldolgozása, egyszerű konvolúcióval.
- S1-2: Mobile Inverted Bottleneck Covolution (MBConv) blokkok. Erőforrásszegény eszközökre tervezett CNN-ekből (EfficientNet, MobileNetV2) átvett ötlet. Először növeli a csatorna számot (1 csatorna 1 kernel kimenete, valamilyen alacsony szintű minta), majd minden csatornán külön futtat 1-1 szűrőt (Deptwise Convolution, Dconv), majd visszacsökkenti a csatornaszámot, hogy megegyezzen a bemenet dimenziójával és összegezni lehessen vele.

- S3-4: Attention blokkok, megtanulja a kép egészére a képrészek közötti kapcsolatokat. Fontos különbség eredeti ViT-het képest, hogy nem abszolút pozíciót rendel hozzá a patch-ekhez, hanem ralatívat (pl.: ettől-attól a patchtől balra/jobbra van ez a patch), hogy koordináták helyett kapcsolatokat tanuljon és így eltolásokra ne legyen érzékeny.
- Kimeneti réteg, átlagolós pooling-al és egy egyszerű neurális hálóval.

  [10]

3 Tervezés

3.1

3.2 Választott adathalmazok

Adathalmaz bemutatása:

https://www.kaggle.com/datasets/wanderdust/skin-lesion-analysis-toward-melanoma-detection

Szintén jónak kinéző dataset:

https://www.kaggle.com/datasets/ahmedxc4/skin-ds?resource=download

A dolgozatomban olyan osztályozót széndákozok létrehozni, amely bőrelváltozásokról állapítja meg azok típusát, kifejezetten, hogy az adott elváltozás melanóma-e, vagy más. Utóbbi esetben egy igen/nem válasz is elég, de az idáig vezető úton több kategória használata és a modellek azon való tanítása és kiértékelése hasznos következtetésekre vezethet. Ezen belül fontos, hogy az egyes kategóriákba tartozó képek egymáshoz képest megfelelő arányban jelen legyenek.

Fontos szempont még az adatok mennyisége. Transzformer típusú hálóknál, ha nulláról tanulnak milliós nagyságrendő képre, de előtanított modellek esetén is ezres nagyságrendű képre van szükség.

A szempontjaimnak a Melanoma Detection Dataset, illetve a Skin Lesions Classification Dataset-t bizonyult ígéretesnek.

Adatelemzés incoming

3.3 Választott technológiák

3.3.1 Számítási platform

Felhű alapú fejlesztői környezetekkel való korábbi tapasztalataim során szembesültem azzal, hogy bár kényelmes a függőségek telepítésének terhétől megszabadulni, a felhasználás mértékének korlátozása miatt könnyű kifutni a számítási keretből még a dolgozahoz

tervezettnél kisebb hálók esetén is. Az indikolja, hogy az orvosi adatokat osztályozó hálókat saját eszközön futassam és tanítsam.

3.3.2 Programnyelv és keretrendszerek

Python, pytorch, torchvision, jupyter notebook-ok

3.3.3 Explainable AI

captum

4 Irodalomjegyzék

[1] Maria Trigka, Elias Dritsas, „A Comprehensive Survey of Machine Learning Techniques and Models for Object Detection," január 2025. [Online]. Available: https://www.mdpi.com/1424-8220/25/1/214 [Hozzáférés dátuma: 27 március 2026].

[2] Mingle Xu, Sook Yoon, Alvaro Fuentes, Dong Sun Park, „A Comprehensive Survey of Image Augmentation Techniques for Deep Learning" november 2022. [Online]. Available: https://arxiv.org/pdf/2205.01491. [Hozzáférés dátuma: 27 március 2026].

[3] IBM, „The 2026 Guide to Machine Learning " 2026. [Online]. Available: https://www.ibm.com/think/machine-learning#605511093. [Hozzáférés dátuma: 05 május 2026].

[4] Wikipedia, „Kernel (image processing)" [Online]. Available: https://en.wikipedia.org/wiki/Kernel_(image_processing). [Hozzáférés dátuma: 06 május 2026].

[5] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, Illia Polosukhin, „Attention Is All You Need" 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf. [Hozzáférés dátuma: 06 május 2026].

[6] Alexey Dosovitskiy,Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby, „AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE" 2021. [Online]. Available: https://arxiv.org/pdf/2010.11929/100. [Hozzáférés dátuma: 07 május 2026].

[7] Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun, „Deep Residual Learning for Image Recognition" december 2015. [Online]. Available: https://arxiv.org/pdf/1512.03385v1. [Hozzáférés dátuma: 10 április 2026].

[8] Dr. Szegletes Luca, „ Geometriai és topologikus mélytanulás” 2026. tavasz.

[9] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, Saining Xie, „A ConvNet for the 2020s” Március 2022. [Online]. Available: https://arxiv.org/pdf/2201.03545. [Hozzáférés dátuma: 14 Április 2026].

[10] Zihang Dai, Hanxiao Liu, Quoc V. Le, Mingxing Tan, „CoAtNet: Marrying Convolution and Attention for All Data Size” szeptember 2021. [Online]. Available: https://arxiv.org/pdf/2106.04803. [Hozzáférés dátuma: 14 Április 2026].

[11] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, Baining Guo, „Swin Transformer: Hierarchical Vision Transformer using Shifted Windows” augusztus 2021. [Online]. Available: https://arxiv.org/pdf/2103.14030. [Hozzáférés dátuma: 14 április 2026].

[x] Szerző, „Cím” június 2009. [Online]. Available: url. [Hozzáférés dátuma: 20 Március 2026].

Függelék

4.1 Függelék

Nyilatkozat generatív mesterséges intelligencia alkalmazásáról[1]

- ☐ **Nem használtam** semmilyen generatív MI segédeszközt.
- ☐ **Használtam** generatív MI segédeszközt. Az MI-vel generált tartalmakat ellenőriztem, a generált kimenetek valóságtartalmáról meggyőződtem, az alábbi táblázatban megfelelően jelöltem minden használatot.

Felhasználási módok	**Generatív MI eszköz(ök) neve**	**Érintett részek** (fejezet, oldalszám, hivatkozás)	**Használat becsült aránya** (felhasználási módonként)
Irodalomkutatás			
Prompt lényegi része			
Programkód generálása			
Prompt lényegi része			
Új ötletek, megoldási javaslatok generálása			
Prompt lényegi része			
Vázlat létrehozása (szövegstruktúra, vázlatpontok)			
Prompt lényegi része			
Szövegblokkok létrehozása			
Prompt lényegi része			
Képek generálása illusztrációs célból			

[1] https://vik.bme.hu/hallgatoknak/altalanos/mi-hasznalat-ajanlasok

Prompt lényegi része			
Adatvizualizáció, grafikonok generálása adatpontok alapján			
Prompt lényegi része			
Prezentáció készítése			
Prompt lényegi része			
Egyéb (nevezze meg)			
Prompt lényegi része			
Összesített százalékos érték (a feladat érdemi részére nézve)			
Összesített érték rövid, szöveges indoklása:			